

Surplus Food Optimization Platform

*Driving Business Efficiency and Social Impact Through
Data-Driven Surplus Food Distribution*

VISHAL JAVVAJI





INDEX

S. No.	Topic	Page No.
1	Introduction	3
2	About Organisation	3
3	Purpose of the System	3
4	Vision – Phase 1 – Cloud Architecture	4
5	Lakehouse Architecture Overview	4 – 7
6	Ingestion Strategy	7 – 9
7	CI/CD Deployment Strategy for Cloud-Based Data Pipeline	9 – 11
8	Pipeline Fail Strategy	11 – 14
9	Monitoring & Alerting	14 – 16
10	Security & Governance	16 - 18
11	Future Enhancements	18
12	Conclusion	19 - 20



I. Introduction

Food waste is a big problem — but with the right tools, it can become an opportunity. Replate is a platform that helps collect and sell surplus food, making sure it gets to people who need it instead of going to waste.

In this project, we built a cloud-based system using Microsoft Azure to manage and track food inventory, deliveries, payments, and customer activity. By bringing all this data together, Replate can make better decisions, improve operations, and reduce waste - all in real time.

II. About Organization

The goal of this organisation was to design a smart data system for **Replate**, a company that sells surplus food online. We used cloud tools like **Azure Data Factory**, **Azure Synapse**, and **Azure Data Lake Storage Gen 2** to collect and manage data from different sources like food partners, delivery trucks, websites, and payments.

We organized the data into three layers:

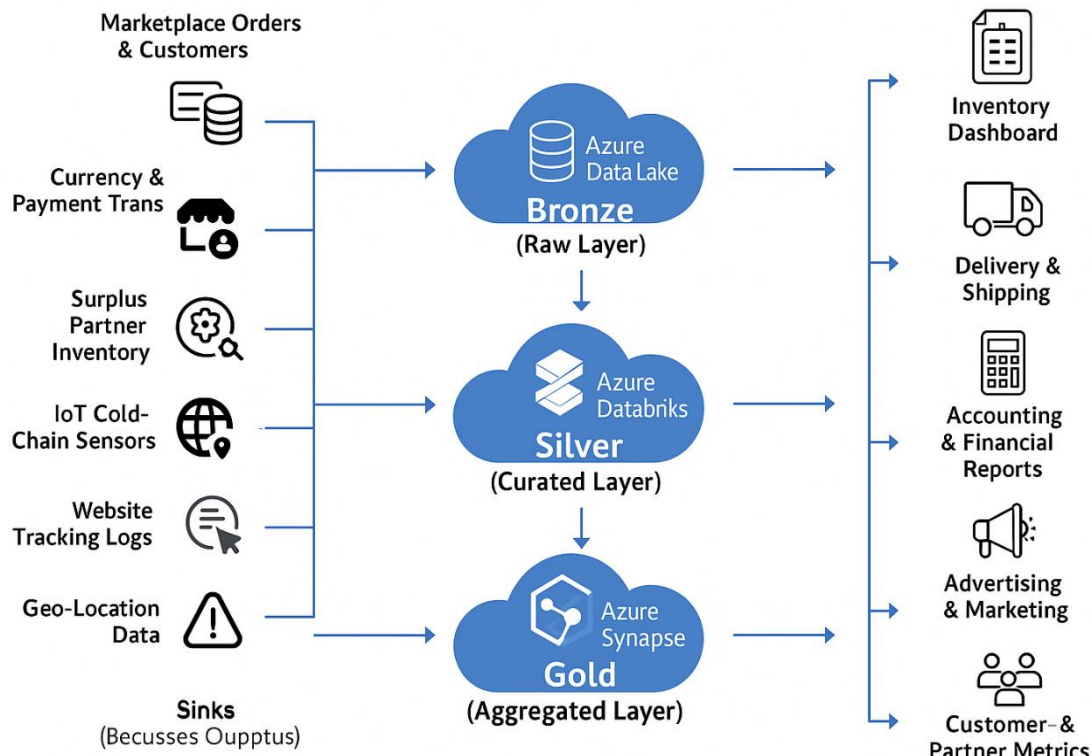
- **Raw (Bronze)** – where we stored data as it came in
- **Cleaned (Silver)** – where we processed and structured the data
- **Final (Gold)** – where the data was ready for reports and dashboards

With this system, Replate can now track inventory, monitor delivery performance, check financials, and make better marketing and business decisions.

III. Purpose of the System

We built a cloud-based data system for Replate to manage surplus food more efficiently. Using Microsoft Azure tools, we collected and organized data from orders, inventory, deliveries, payments, and website activity. This helped Replate track performance, reduce waste, and make faster business decisions through clear dashboards and reports.

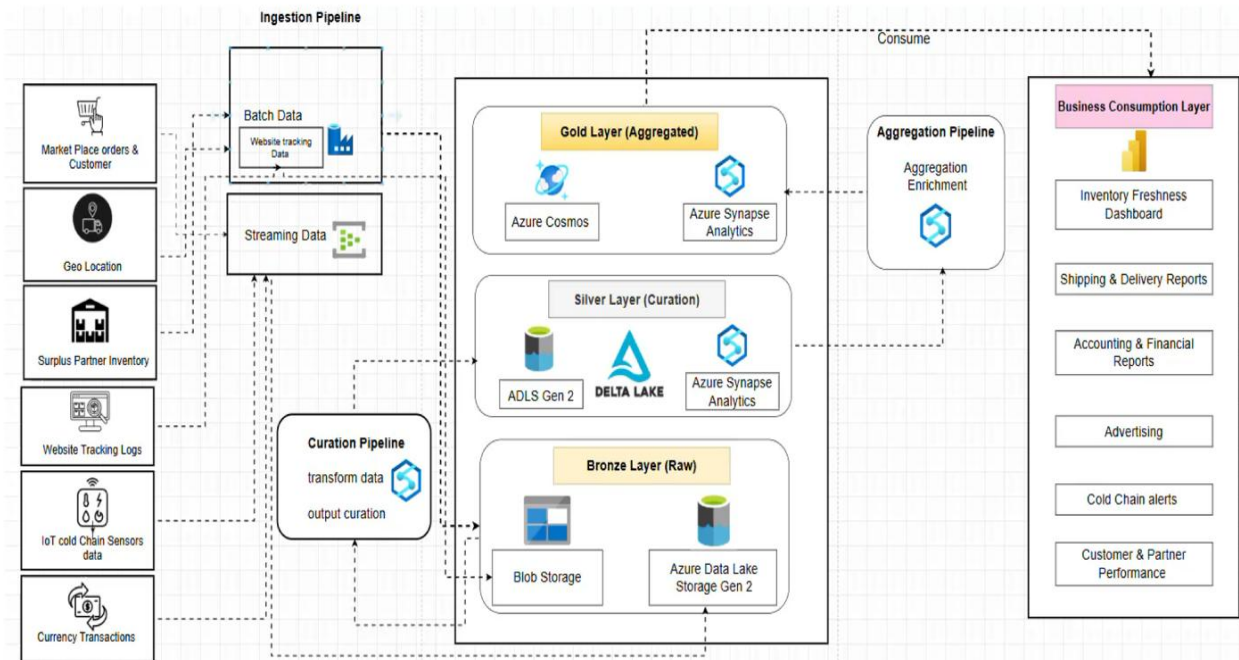
IV. Vision – Phase 1 – cloud architecture



- **Replate's mission** is to ensure that no surplus food goes to waste. To do that, we use cloud technology to collect, clean, analyze, and deliver insights from various data sources — helping improve food recovery, logistics, partner performance, and customer satisfaction.
- Our vision is to **transform food waste into meaningful action** through data — enabling timely decisions, better planning, and measurable impact across inventory, delivery, safety, and community engagement.
- This vision is powered by a modern **Lakehouse Architecture** on Azure, which streamlines our data journey from collection to consumption.

V. Lakehouse Architecture Overview

The architecture is built using three layered stages: Bronze (raw), Silver (curated), and Gold (aggregated), each using specific Azure services for ingestion, transformation, and analytics.



1. Bronze Layer – Raw Data Storage

- **Platform:** Azure Data Lake
- **Purpose:** This is where all data is ingested and stored in its original/raw format. It allows us to retain full data history and revisit the source data anytime.

▼ Data Sources flowing into this layer:

- **Marketplace Orders & Customers** – Contains customer and order history.
- **Currency & Payment Transactions** – Includes payment gateways and refunds (via streaming).
- **Surplus Partner Inventory** – Information from partners about food stock levels and freshness.
- **IoT Cold-Chain Sensors** – Real-time data from storage and transport devices to monitor food safety.
- **Website Tracking Logs** – Clickstream data and behavior analytics from our site.
- **Geo-Location Data** – Delivery zones and IP-location mappings.

All these sources flow into Azure Data Lake (Bronze) via ingestion tools like:

- **Azure Data Factory (ADF)** – for batch loads



- **Azure Event Hubs** – for real-time streaming
This ensures we capture data consistently and reliably.

2. Silver Layer – Curated Data for Analysis

- **Platform:**
 - ADLS Gen2 for storage
 - Delta Lake for versioned and transactional data
 - Azure Synapse Analytics for querying and analysis
- **Purpose:**
Clean and structure data to make it ready for business reporting and insights.
- **Activities:**
 - Cleans and validates raw data (e.g., removes duplicates, fills missing values)
 - Applies filters to select useful records for analysis
 - Enriches data by joining sources (e.g., orders + customer info)
 - Implements logic like refund %, delivery delays, inventory flags
- **Flow:**
 - **Input:** Takes raw data from Bronze Layer (Blob + ADLS Gen2)
 - **Processing:** Transforms data using Delta Lake (via ADF or Synapse pipelines)
 - **Output:** Stores curated datasets back in ADLS Gen2 (Delta format) and makes it queryable by Azure Synapse Analytics
 - **Feeds:** Sends to Gold Layer for aggregation and final consumption

3. Gold Layer – Aggregated Data for Business Intelligence

- **Platform:**
 - Azure Synapse Analytics for querying
 - Azure Cosmos DB for scalable aggregation and enrichment
- **Purpose:**
Deliver high-value, aggregated insights that power business dashboards and reports.
- **Activities:**
 - Combines curated data from Silver Layer for summary-level reporting



- Performs aggregations (e.g., total orders per region, average delivery time)
- Enriches datasets with metrics (e.g., freshness scores, partner performance)
- Optimizes for fast querying in the business consumption layer
- **Flow:**
 - **Input:** Receives curated datasets from Silver Layer
 - **Processing:** Aggregation and enrichment via Azure Synapse + Cosmos DB
 - **Output:** Serves aggregated data to Power BI dashboards, financial reports, shipping summaries, and performance metrics in the Business Consumption Layer

Business Outputs (Sinks)

The final insights from the Gold (and sometimes Silver) layer are used to power the following dashboards and reports:

- **Inventory Dashboard** – Tracks freshness, expiry, and stock levels.
- **Delivery & Shipping** – Reports on delivery success and delays.
- **Accounting & Financial Reports** – Shows revenue, payment, and refund trends.
- **Advertising & Marketing** – Tracks website engagement and campaign success.
- **Customer & Partner Metrics** – Evaluates satisfaction, repeat orders, and partner reliability.

VI. Ingestion Strategy

A well-designed ingestion strategy is critical to building a scalable and reliable cloud architecture. In the Replate project, data ingestion acts as the foundation that feeds multiple data sources into our centralized storage and processing pipeline. Since the data comes from both batch and streaming systems, we used a hybrid ingestion approach to ensure timeliness, completeness, and flexibility.

Purpose of Ingestion

The primary goal of our ingestion layer is to collect raw data from diverse internal and external systems—such as online marketplace orders, IoT devices, partner inventories, and user interactions—and land them into the Bronze Layer (Raw Zone) of our Azure-based data lake in an organized and secure way. From here, the data becomes accessible for downstream curation and analysis.



Ingestion Tools Used

To handle the variety of data sources and formats, we used the following Microsoft Azure services:

- **Azure Data Factory (ADF):** For scheduled batch ingestion from databases, APIs, CSVs, and log files. ADF orchestrates and automates data movement.
- **Azure Event Hubs:** For real-time streaming ingestion, especially from IoT cold-chain sensors and payment gateways.
- **Azure Blob Storage:** Used for landing log files (like website tracking data) before moving to the data lake.
- **REST APIs:** ADF interacts with external APIs (e.g., geo-location, inventory) to pull structured and semi-structured data.
- **Azure Data Lake Storage Gen2 (ADLS Gen2):** Destination for all ingested data into the Bronze Layer, with proper folder and time-based partitioning.

How Ingestion Works (Step-by-Step)

Each data source is handled with its own ingestion flow, designed around its format, frequency, and business criticality:

1. Marketplace Orders & Customers

- **Type: Batch (Structured)**
- **Method:** ADF pulls from SQL databases using incremental loads (Change Data Capture)
- **Frequency:** Hourly
- **Target:** Stored in /bronze/orders/yyyy/mm/dd/ in ADLS Gen2

2. Currency & Payment Transactions

- **Type:** Batch + Streaming (Semi-structured)
- **Method:** Real-time logs from Event Hubs; hourly settlement data via ADF
- **Target:**
 - **Streaming** → /bronze/payments/streaming/
 - **Batch** → /bronze/payments/settlements/

3. Surplus Partner Inventory

- **Type:** Batch (Structured)
- **Method:** ADF connects to partner ERP systems or APIs



- **Frequency:** Hourly
- **Target:** /bronze/inventory/yyyy/mm/dd/

4. IoT Cold-Chain Sensors

- **Type:** Streaming (Semi-structured JSON)
- **Method:** Real-time ingestion via Event Hubs
- **Frequency:** Continuous
- **Target:** /bronze/iot_sensors/yyyy/mm/dd/

5. Website Tracking Logs

- **Type:** Batch (Unstructured)
- **Method:** Logs land in Blob storage, then ADF moves them into the lake
- **Frequency:** Hourly or Daily
- **Target:** /bronze/web_logs/yyyy/mm/dd/

6. Geo-Location Data

- **Type:** Batch (Structured or CSV)
- **Method:** ADF calls REST API or downloads files
- **Frequency:** Daily
- **Target:** /bronze/geo_location/yyyy/mm/dd/

VII. CI/CD Deployment Strategy for Cloud-Based Data Pipeline

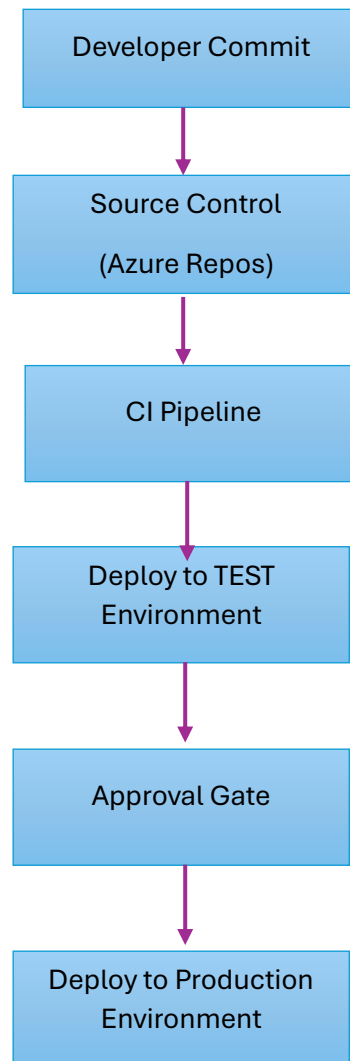
In a modern data engineering setup like ours, CI/CD (Continuous Integration and Continuous Deployment) plays a vital role in automating the lifecycle of data pipelines—from development to deployment. This ensures that changes made to the ingestion, transformation, or aggregation layers are quickly tested, reviewed, and delivered into production without manual intervention, reducing risk and increasing speed.

- **Purpose of CI/CD in This Project**
 - Enable faster iterations on pipeline development (Ingestion, Curation, Aggregation).
 - Minimize deployment errors and ensure version control.
 - Automatically test and validate changes before pushing them to production.
 - Support multiple environments (Dev → Test → Prod) for safe deployments.

- **Tools Used in Our Architecture**

Stage	Tools Used
Version Control	Azure Repos
Build & Pipeline Tool	Azure DevOps Pipelines
Infrastructure as Code	ARM templates
Triggering CI/CD	Scheduled & Event Trigger

- **CI/CD Flow in Our Architecture**





1. Developer Commit

- A data engineer or developer pushes changes (e.g., new pipeline logic or script updates) to the code repository.

2. Source Control Azure Repos

- The code is stored in a version-controlled system like Azure Repos.
- This ensures tracking, rollbacks, and team collaboration.

3. CI Pipeline (Continuous Integration)

The CI pipeline automatically runs the following checks:

- **Linting & Validation:** Checks pipeline definitions (JSON/ARM templates) for errors or misconfigurations.
- **Unit Testing:** Tests core logic in scripts—like Python functions, ADF pipeline scripts, or SQL views.
- **Static Code Analysis:** Ensures code quality and follows best practices.

4. Artifact Build

- Once validated, the tested version of the pipeline (build) is packaged as an artifact.
- This is stored in tools like Azure DevOps Artifacts or GitHub Packages, ready for deployment.

5. CD Pipeline (Continuous Deployment)

The CD pipeline takes the validated artifact and begins automated deployment across environments.

❖ Deploy to Development (DEV)

- The pipeline is first deployed in a safe development environment.
- Internal tests validate ingestion, transformation, and load logic.

❖ Approval Gate

- After successful testing, the deployment waits for manual review or approval to move forward.

❖ Deploy to Testing (TEST)

- In some cases, pipelines are promoted to a staging environment to perform more integrated tests with mock or limited live data.

❖ Final Approval Gate

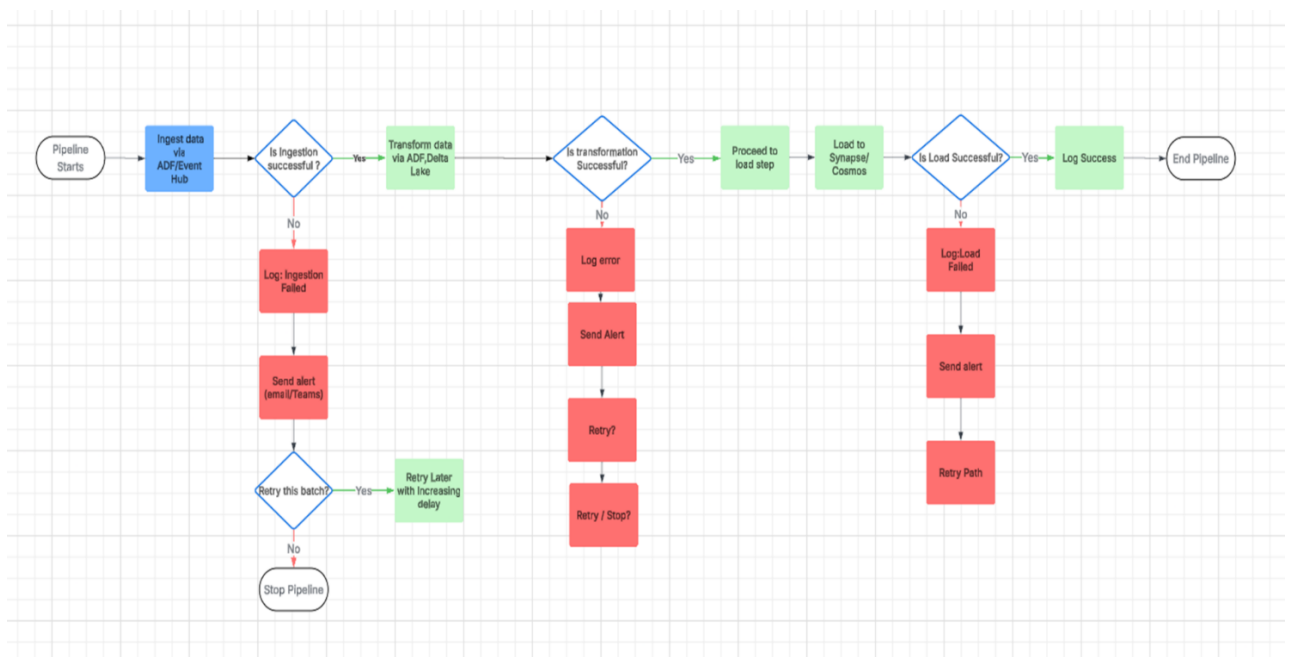
- A final manual check ensures everything is in order before production rollout.

❖ Deploy to Production (PROD)

- The final deployment rolls out:
 - Azure Data Factory pipelines (ingestion & curation)
 - Azure Synapse SQL scripts
 - Delta Lake definitions
 - Power BI reports (if included)

VIII. Pipeline Fail Strategy

A well-structured pipeline failure strategy is essential in any modern data architecture to ensure that data ingestion, transformation, and loading processes are resilient, transparent, and easily recoverable. This strategy outlines how failures at each stage are detected, logged, and handled—whether by retrying, alerting teams, or stopping the process entirely to prevent downstream impact.





1. Pipeline Start

The process begins when a trigger (event-based or scheduled) initiates the pipeline to move data from source systems into the data platform.

2. Ingestion Stage – ADF / Event Hub

- **Process:** Data is ingested either in real-time (via Azure Event Hub) or in batch (via Azure Data Factory).
- **Checkpoint:** The pipeline checks “Is Ingestion Successful?”

✓ If Successful:

- The process continues to the transformation stage.

✗ If Failed:

- The system:
 - Logs the error ("Ingestion Failed")
 - Sends an alert via email or Microsoft Teams
 - Prompts a decision: Retry this batch?
 - If Yes → Retry with increasing delay (Exponential Backoff).
 - If No → Gracefully stop the pipeline and avoid corrupt downstream flows.

This ensures ingestion errors (e.g., unreachable endpoints, malformed files) are caught early without impacting later stages.

3. Transformation Stage – ADF + Delta Lake

- **Process:** Raw ingested data is cleaned, joined, and business rules are applied using ADF Data Flows and Delta Lake.
- **Checkpoint:** The pipeline checks “Is Transformation Successful?”

✓ If Successful:

- Proceed to the next step: loading data.

✗ If Failed:

- The system:
 - Logs the error ("Transformation Failed")
 - Sends an alert (email/Teams)
 - Asks whether to retry or not:
 - If Retry, it will reattempt transformation logic.



- If it repeatedly fails, the pipeline decides whether to stop or escalate.

Transformation failures (like schema mismatches, NULL propagation, or logic errors) are caught before reaching analytics layers.

4. Load Stage – Synapse or Cosmos DB

- **Process:** Transformed data is loaded into target destinations such as Azure Synapse Analytics (for querying) or Cosmos DB (for real-time applications).
- **Checkpoint:** “Is Load Successful?”

✓ If Successful:

- The system logs a successful execution and completes the pipeline.

✗ If Failed:

- The system:
 - Logs the error ("Loading Failed")
 - Sends an alert to the engineering team
 - Attempts a Retry Path (configured retries + alert escalation)

This prevents invalid or partial data from reaching business reports and dashboards.

5. Pipeline Ends

If all stages pass, the pipeline logs success and ends gracefully. This allows downstream tools (like Power BI or reporting APIs) to work with clean and reliable data.

IX. Monitoring & Alerting

In a modern data architecture—especially those that include hybrid ingestion through Azure Data Factory (ADF) and Azure Event Hubs—it is essential to integrate real-time monitoring and alerting to ensure pipeline observability, data reliability, and rapid incident response. The architecture in reference demonstrates a comprehensive failure-handling mechanism across ingestion, transformation, and loading stages. However, the true value of such fault tolerance comes from its monitoring and alerting strategy, which is discussed below in depth.

Purpose of Monitoring & Alerting

The goal of monitoring and alerting in this pipeline architecture is to:

- Detect failures (ingestion, transformation, or load) in near-real-time.
- Capture error context for logs and future root cause analysis.
- Notify relevant stakeholders (developers, ops teams) for quick action.
- Trigger automated retries or controlled shutdowns based on thresholds.



Where Monitoring Happens in the Pipeline

From the image provided, monitoring and alerting are embedded in **each critical decision point**:

1. Ingestion Stage

- *Condition checked*: Is ingestion successful?
- If failed:
 - Error is logged.
 - Alert is sent via email or Teams.
 - Retry logic is conditionally triggered.

2. Transformation Stage

- *Condition checked*: Is transformation successful?
- If failed:
 - Logs are captured (e.g., bad joins, nulls, schema mismatches).
 - Alert sent immediately to notify failure in transformation logic.
 - Retries are evaluated based on pre-set conditions.

3. Load Stage

- *Condition checked*: Is load to Synapse or Cosmos successful?
- If failed:
 - Log entry made with timestamp and error code.
 - Alert triggered with contextual info.
 - Retry mechanism attempts controlled re-execution.

Implementation Tools for Monitoring & Alerting

To support this monitoring system in Azure, the following services can be used:

Tool	Purpose
Azure Monitor	Centralized metrics and logs across resources (ADF, Event Hub, Synapse, etc.)
Log Analytics	Query and analyze logs for pipeline execution trends and error patterns
Application Insights	Monitor transformation scripts or APIs involved in the process



Tool	Purpose
Alerts in ADF	Triggers alert rules on activity failures or timeout events
Azure Logic Apps / Azure Functions	For automating custom alert workflows (e.g., sending Teams notifications)
Power BI Dashboards	Visualizing pipeline success/failure trends, retry rates, and latencies

Alerting Mechanisms

Alerts are configured to notify on:

- Activity failure (ADF, Event Hub consumer groups)
- Exceeded latency thresholds
- Missing or delayed pipeline runs (based on schedule)
- Retry count exceeded
- Data volume anomalies (e.g., too few records loaded)

Alert destinations can include:

- Email Distribution Lists
- Microsoft Teams Channels
- ServiceNow or Jira (for automatic ticket creation)
- PagerDuty/Slack (for DevOps on-call notifications)

Best Practices

- Set retry thresholds to avoid infinite loops and noisy alerts.
- Enrich alerts with pipeline name, activity ID, timestamp, and error message.
- Log all failures centrally for easy audit and RCA.
- Use dashboards to monitor pipeline KPIs like success rate, average duration, and retries.
- Test alerting flows regularly to ensure the right people are notified.



X. Security & Governance

Assigns and tracks work by employee. Useful for:

- Supervisors to monitor staff activity.
- Ensuring no employee is overburdened.
- Improving accountability.

In our data pipeline architecture, ensuring data protection, compliance, and traceability is central. Security and governance are tightly embedded across every stage—from ingestion to business consumption—leveraging Azure-native tools and best practices.

1. Role-Based Access Control (RBAC)

- **Purpose:** To ensure only authorized users and services can access data and resources.
- **Implementation:**
 - Data access permissions are configured in Azure Data Lake Storage Gen2, Synapse, and Cosmos DB using Azure RBAC.
 - Granular control ensures read-only access to analytics teams while write privileges are limited to transformation pipelines (ADF, Delta Lake).

2. Data Encryption

- **At-rest:** All data stored in Blob Storage, ADLS Gen2, and Cosmos DB is encrypted using Azure-managed keys (with optional customer-managed keys for sensitive datasets).
- **In-transit:** Data moving between Event Hub → ADF → Synapse is encrypted via HTTPS and TLS.

3. Data Lineage & Auditing

- **Purpose:** To track data origin, transformations, and movement for transparency.
- **Implementation:**
 - Azure Purview or Synapse Data Catalog can be used to capture lineage from Bronze (raw) to Gold (aggregated).
 - Logs from ADF pipelines are stored centrally, detailing each step, timestamp, and execution metadata.
 - Facilitates easy audits and helps explain insights back to raw sources when required.

4. Monitoring and Alerts

- Pipeline failures, retries, and success logs (as shown in your Pipeline Failure Strategy image) are centrally monitored using:



- Azure Monitor
- Log Analytics
- Alerts via Azure Alerts / Email / Teams / Slack
- This provides visibility and rapid action on anomalies or suspicious behaviors.

5. Data Governance Policies

- Retention Policies: Implemented at the ADLS Gen2 level for raw data to manage storage lifecycle.
- **PII Masking & Compliance:**
 - Sensitive data fields (e.g., customer addresses or transaction amounts) are masked or encrypted.
 - GDPR/CCPA compliance is ensured using Data Masking and Access Audits.

Summary Table for Governance Coverage

Layer	Security Measures
Ingestion	Event Hubs secured with private endpoints & policies
Storage	ADLS Gen2 with RBAC, data masking, encryption
Curation	Data flows validated & logged in ADF
Analytics	Synapse governed with workspace-level security
Consumption	Power BI uses row-level security (RLS) to restrict data views
Monitoring	Log Analytics + Alerts for real-time governance feedback

XI.Future Enhancements

1. Cross-Region Data Replication

- **Why:** To ensure business continuity and disaster recovery.
- **What to Do:** Use Azure Data Factory's integration runtime and ADLS Gen2 replication to mirror data across regions, enabling failover during regional outages.

2. Event-Driven Microservices Integration

- **Why:** To decouple services and trigger business workflows in real time.



- **What to Do:** Use Azure Functions or Logic Apps to respond to Event Hub messages (e.g., notify warehouse when inventory falls below threshold).

3. No-Code/Low-Code Data Exploration for Analysts

- **Why:** To empower business users and reduce developer bottlenecks.
- **What to Do:** Enable Power BI Dataflows and Azure Synapse Serverless SQL Pools for ad-hoc querying over curated Silver/Gold layer datasets.

4. Data Retention & Tiered Archiving Policy

- **Why:** To optimize storage costs and comply with data regulations.
- **What to Do:** Implement lifecycle management policies on ADLS Gen2 to move older Bronze/Silver data to cold or archive storage after X months.

5. Personalized Dashboards via Row-Level Security

- **Why:** To provide partner- or department-specific KPIs securely.
- **What to Do:** Configure Power BI Row-Level Security (RLS) based on user roles or customer IDs embedded in Azure AD.

XII. Conclusion

This project successfully delivers a scalable, modular, and secure cloud-based data platform built on the Azure ecosystem, designed to handle both batch and real-time data ingestion, curated processing, and business-ready reporting. The architecture integrates modern data engineering principles to support high-volume transactional systems such as marketplace orders, customer behavior tracking, inventory, and cold chain logistics.

Key Achievements

- **Hybrid Ingestion Architecture:**
We implemented a dual-ingestion strategy using Azure Data Factory for batch pipelines (e.g., customer data, geolocation, web logs) and Azure Event Hubs for real-time streaming (e.g., currency transactions, IoT sensor feeds), ensuring flexible and timely data flow from diverse sources.
- **Multi-Layered Lakehouse Model:**
The system follows a **Bronze-Silver-Gold architecture**:
 - Bronze Layer captures raw data in Azure Data Lake Gen2 and Blob Storage.
 - Silver Layer performs transformation and curation using Delta Lake and ADF pipelines.
 - Gold Layer aggregates and models data for analytics using Synapse and Cosmos DB.



- **Pipeline Design & Fault Strategy:**
 - A robust pipeline failure strategy is in place with error logging, alerting (email/Teams), retry logic, and escalation paths at each stage — ingestion, transformation, and load.
 - Custom retry delays and decision logic ensure the pipeline remains resilient under failure scenarios.
- **CI/CD & Deployment Strategy:**
 - CI/CD flows were implemented using GitHub and Azure DevOps, incorporating validation, unit testing, packaging, and gated approvals.
 - Seamless promotion from development to staging to production environments maintains release integrity and auditability.
- **Security & Governance:**
 - End-to-end data lineage, monitoring, and role-based access controls were implemented.
 - Azure Purview (optionally extendable) can be used for future governance and metadata management.
- **Business Value Delivery:**
 - Final data outputs serve key business layers including:
 - Inventory dashboards
 - Shipping reports
 - Advertising insights
 - Customer/partner performance metrics